

Agentic RAG for HKEX Listing Rules Compliance

1st Interim Report

Student Name: Li Yubo

Student ID: 59903650

Supervisor: Chen MA

Course: CS6520

Date: 2026.4.6

Table of Contents

1. Introduction

1.1 Background and Problem Context

1.2 Project Objectives

1.3 Practical Value and Expected Outcome

2. Related Work

2.1 Traditional Information Retrieval for Regulatory Documents

2.2 Neural and Dense Retrieval Methods

2.3 RAG and Agentic RAG Systems

2.4 Relationship Between Existing Work and This Project

3. System Modeling and Structure

3.1 Problem Scope and System Boundary

3.2 Overall Architecture

3.3 LangGraph Workflow Structure

3.4 Design Justifications

4. Methodology and Algorithms

4.1 Knowledge Base Construction Pipeline

4.2 Structure-Aware Chunking Algorithm

4.3 Hybrid Retrieval Strategy

4.4 Agentic Workflow: Planning, Retrieval and Reasoning

4.5 Current Limitations and Planned Enhancements

5. Preliminary Performance Analysis or Experiments

5.1 Experimental Setup

5.2 Functional Verification

6. Milestones and Overall Schedule

6.1 Work Completed So Far

6.2 Project Schedule and Milestones

7. Work to be Completed for the Next Report

7.1 Technical Improvements

7.2 Evaluation and Reporting Work

8. References

1. Introduction

1.1 Background and Problem Context

The Hong Kong Stock Exchange (HKEX) is a major global financial market, with over 2,600 listed companies and a market capitalization of more than HKD 30 trillion as of 2025 [1]. Companies listed on the exchange must follow the HKEX Listing Rules. The Main Board Listing Rules contain over 30 chapters and more than 2,500 specific rules, plus extensive appendices and guidance materials [2]. The Growth Enterprise Market (GEM) has a similar rulebook. HKEX also publishes supplementary materials including Guidance Letters, Listing Decisions, and Frequently Asked Questions to help interpret these rules [3].

These documents are heavily cross-referenced. A company trying to understand the requirements for a connected transaction might need to check Chapter 14A (Connected Transactions), Chapter 14 (Notifiable Transactions), Chapter 2 (Definitions), and several guidance letters. Each rule can point to other sub-rules or exceptions. Company secretaries and legal advisors often struggle to locate all relevant provisions and reach a clear conclusion [4].

Traditional keyword search can find specific rules, but it fails when dealing with this complexity. It cannot understand legal terminology, handle multi-step reasoning, or identify implicit cross-references between chapters. Practitioners spend considerable time manually tracing these connections, which increases the risk of missing important obligations. This motivates the need for more intelligent tools that can provide accurate and evidence-based answers to compliance questions, such as Retrieval-Augmented Generation (RAG) systems with agent planning capabilities.

1.2 Project Objectives

This project builds an Agentic Retrieval-Augmented Generation (RAG) system for HKEX Listing Rules compliance questions. Standard RAG systems search and retrieve information once. Our system adds agentic planning and multi-step reasoning to handle the complex structure of regulatory documents [5].

The primary objectives for Phase 1 are:

1. **Document Ingestion:** Import HKEX documents (Main Board, GEM, and guidance materials), perform structure-aware chunking that preserves rule numbers and chapter titles, and build a search index using both keyword (BM25) and semantic (embedding) methods [6].
2. **Agentic Workflow Prototype:** Build a workflow using LangGraph with three components: a Planner to classify questions, a retriever to find relevant information, and a reasoning component to synthesize answers. The system handles both simple rule lookups and complex questions that require combining multiple rules [7].
3. **Citation-Grounded Answers:** Every answer must be supported by evidence from the rules. Each answer includes clear citations with rule numbers and chapter titles, which is necessary for compliance tasks where evidence must be verifiable [8].
4. **Backend API:** Develop a FastAPI backend with a modular architecture. This makes it easier to add new features later, such as specialized compliance tools, frontend interfaces, or evaluation datasets [9].

Phase 1 does not include: a web interface, specialized calculators, a full benchmark dataset, formal evaluation frameworks, or production-level deployment. We focus on building a functional backend prototype to validate the approach.

1.3 Practical Value and Expected Outcome

Compliance officers, company secretaries, and legal advisors often spend hours searching for information across multiple chapters and guidance documents. This system can help by automatically locating relevant rules, tracing cross-references, and providing cited answers. This reduces both time spent and the risk of missing important obligations [10].

The project also explores how information systems can reason about legal documents instead of just matching keywords. This could support more advanced tools in the future, such as automated disclosure checkers or scenario-based advisory systems [11].

Phase 1 aims to deliver:

1. **Functional Backend Prototype:** A Python application that can ingest HKEX documents, index them, and provide compliance answers via a RESTful API.

2. Knowledge Pipeline: A structured workflow that processes regulatory documents, preserves their hierarchy, and enables precise retrieval.
3. Preliminary Validation: Initial testing to prove that the system can handle both simple and multi-hop compliance questions and return answers backed by traceable sources.
4. Architectural Foundation: A modular design that allows for easy integration of new tools, frontends, and evaluation methods in the next phase [12].

2. Related Work

To understand how our system fits into the broader landscape of compliance question answering, we review four areas of related work. We begin with traditional information retrieval methods, which provide the foundation for keyword-based search. We then discuss neural and dense retrieval approaches, which improve semantic recall. Next, we examine the evolution from standard RAG to Agentic RAG systems. Finally, we position our work relative to these existing approaches and acknowledge the current limitations of our Phase 1 prototype.

2.1 Traditional Information Retrieval for Regulatory Documents

Traditional information retrieval systems, such as keyword-based search and the BM25 algorithm, have long been the backbone of regulatory document retrieval [12, 13]. These systems rely on inverted indexes to match user queries with terms appearing in the document corpus.

The main advantage of these methods is their stability and speed. Because they are based on literal term matching, they provide high interpretability; users can clearly see why a specific document was retrieved based on the occurrence of their search terms. For many legal and compliance tasks, this deterministic nature is a significant benefit, as it minimizes the risk of the system retrieving irrelevant content when no keyword match exists, though it risks missing relevant documents that use different wording.

However, traditional IR systems have critical limitations for complex regulatory scenarios like the HKEX Listing Rules. First, they lack semantic understanding. A query like "disclosure requirements for connected transactions" may fail to retrieve relevant documents if the documents use synonyms or related terms not explicitly present in the query. Second, they struggle with multi-hop questions. When a query requires synthesizing information from multiple provisions — for instance, checking both disclosure thresholds and shareholder approval rules — a keyword-based system can only retrieve individual fragments, leaving the user to manually piece them together. This limitation often requires multiple rounds of query refinement, which is time-consuming and prone to error.

2.2 Neural and Dense Retrieval Methods

To address the limitations of keyword matching, researchers have increasingly turned to dense retrieval methods using neural networks [14]. These approaches map queries and documents into a shared continuous vector space, where semantic similarity is measured by distance (e.g., cosine similarity).

Dense retrieval excels at capturing semantic relationships. By representing the underlying meaning of text, models like BERT [15] or more specialized embedding models such as BGE can retrieve documents that are semantically relevant even if they share no common keywords with the query. When paired with vector databases like FAISS or Chroma, these systems can perform fast approximate nearest neighbor searches on large datasets.

Despite this advantage, dense retrieval has its own drawbacks in legal domains. Because the matching process relies on learned embeddings, it can occasionally retrieve semantically similar content that is legally imprecise for the specific query. In high-stakes environments like HKEX compliance, where the precision of rule interpretation matters, semantic similarity does not always guarantee legal correctness. A retrieved chunk might be about connected transactions but actually describe an unrelated exemption, which could mislead the system into suggesting an incorrect rule.

2.3 RAG and Agentic RAG Systems

Retrieval-Augmented Generation (RAG) improves upon pure LLM generation by grounding answers in external, retrieved evidence [16]. A standard RAG pipeline typically involves a single-pass retrieval followed by a generation step. While effective for simple fact retrieval, it remains passive and often inadequate for complex compliance questions that require reasoning across multiple documents.

Agentic RAG represents a shift toward more autonomous systems [17]. Unlike a standard RAG pipeline, an Agentic RAG system uses a planner to break down a user's question, execute multiple rounds of retrieval, and synthesize evidence in a reasoned way. This is particularly useful for legal and regulatory domains. For example, if a compliance query needs clarification, an agent can automatically perform a second round of targeted searching. Frameworks like LangChain and LangGraph facilitate this by allowing the definition of stateful workflows, where the system maintains context and makes iterative decisions about what information is still needed [18].

Recent research shows that this agentic paradigm is better suited for complex Q&A because it shifts the system from a one-shot responder to an iterative reasoner. By separating the roles of planning, retrieval, and reasoning, these systems can check whether they have sufficient evidence before generating a final answer. If the agent determines that the evidence is insufficient or contradictory, it can choose to backtrack or rephrase the query. This degree of control is essential for accurate compliance advising, where a single missed provision can lead to incorrect guidance.

2.4 Relationship Between Existing Work and This Project

Our system integrates both traditional and neural retrieval. We use a hybrid approach that combines BM25 for precise keyword matching and dense embeddings for semantic recall. This balances the need for lexical exactness with the ability to handle synonyms and paraphrased queries.

Beyond retrieval, our system extends standard RAG by introducing a planner and a conditional router. This allows the system to distinguish between simple rule lookups and multi-hop compliance questions, triggering a second retrieval step only when necessary. Each answer includes citations with rule numbers and source document identifiers, which is a requirement for compliance use cases where answer provenance must be traceable.

That said, the current prototype has clear limitations. The planner relies on simple heuristic classification rather than full LLM-driven reasoning. The system does not yet support tool use (such as financial calculators), session memory, or formal evaluation. These are planned for Phase 2. At this stage, the system is best understood as a domain-specific Agentic RAG prototype for HKEX compliance — more capable than a standard single-pass RAG system, but still an early-stage implementation focused on validating the core agentic workflow.

3. System Modeling and Structure

Building on the approaches discussed in Section 2, this section defines the scope, architecture, and workflow of our system.

3.1 Problem Scope and System Boundary

This project focuses on a specific subset of the HKEX Listing Rules: Notifiable Transactions (Chapter 14), Connected Transactions (Chapter 14A), Size Tests (Rule 14.07 and related provisions), and associated disclosure obligations. We chose these areas because they come up frequently in compliance questions and involve cross-referencing between multiple rule provisions.

Table 1: System Input/Output Specification

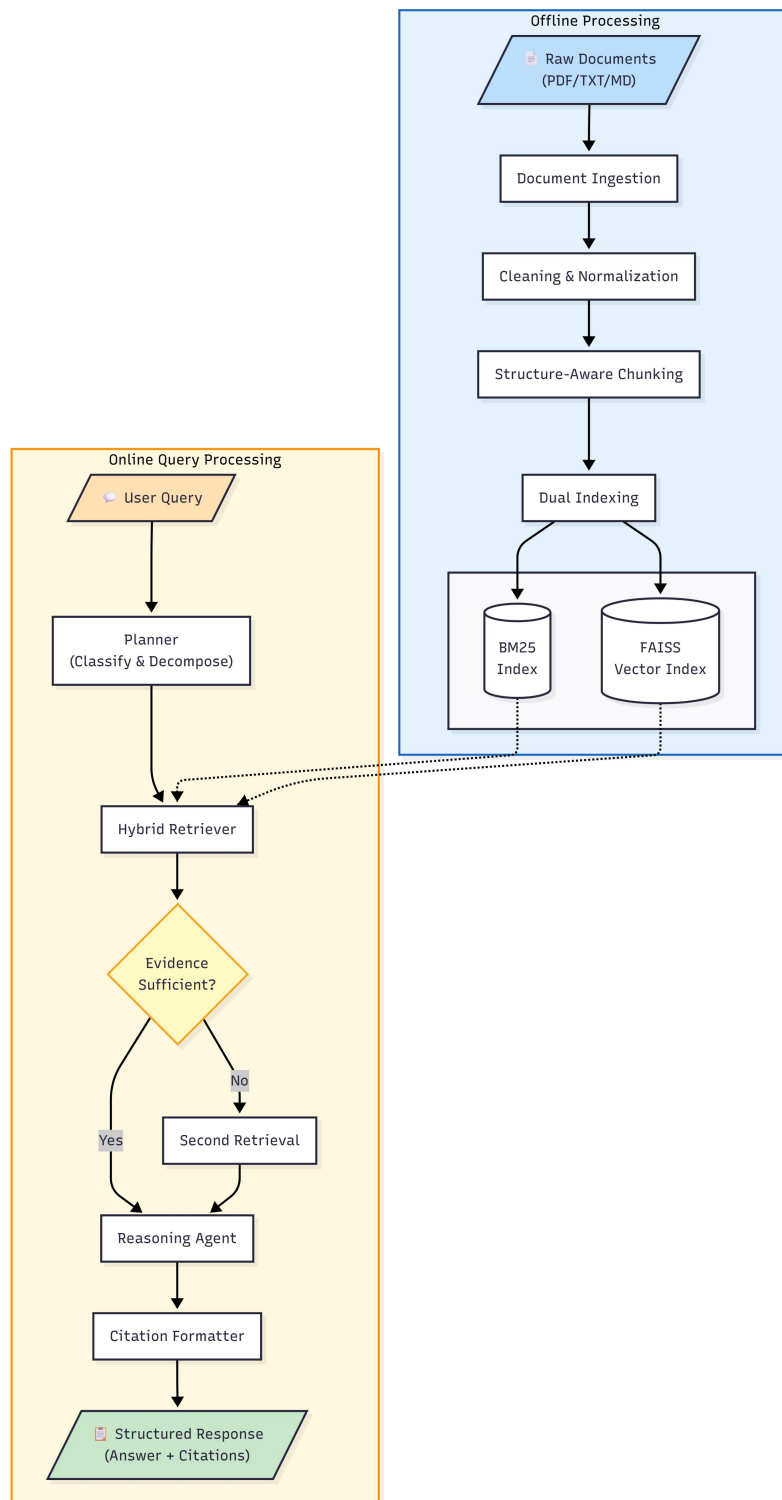
	Description	Format
Input	Natural language compliance question	JSON: {"query": "..."} "
Output - Answer	Natural language response synthesizing relevant rules	string
Output - Citations	Rule numbers, chapter references, source identifiers	List[Citation]
Output - Evidence	Retrieved chunks used as supporting evidence	List[Chunk]
Output - Uncertainty	Note indicating incomplete or conflicting evidence	string or null

For example, a user might ask: "What are the disclosure requirements for a connected transaction?" The system returns a structured JSON response containing all four output components.

3.2 Overall Architecture

The system follows a pipeline with six stages: Document Ingestion, Cleaning, Chunking, Indexing, Query Processing, and Response Generation. Figure 1 shows the overall architecture.

Figure 1 Required: System Architecture Diagram



The first four modules run offline during document preparation. Document Ingestion supports plain text, Markdown, and PDF, though PDF support is basic in Phase 1. Cleaning normalizes text by removing excessive whitespace, fixing encoding issues, and standardizing line breaks. Chunking preserves document structure: instead of splitting at arbitrary token boundaries, the

chunker identifies rule numbers, section titles, and chapter markers, then creates chunks that keep these elements intact. Indexing builds two retrieval indexes: BM25 for keyword search and FAISS for semantic search using BGE-M3 embeddings.

The last two modules run online during query time. Query Processing routes the user's question through the LangGraph workflow described in Section 3.3. Response Generation synthesizes the answer using DeepSeek Reasoner and formats citations with traceable rule numbers.

3.3 LangGraph Workflow Structure

Query processing uses LangGraph to implement a stateful agent workflow. Unlike a hardcoded pipeline, LangGraph allows conditional branching based on intermediate results. Figure 2 shows the workflow.

The workflow starts at the **Planner Node**, which classifies the query as `direct` (simple rule lookup) or `multi_hop` (requires combining multiple rules). For multi-hop queries, the Planner also breaks the question into sub-queries. Classification uses heuristic rules based on keyword patterns, checking for conjunctions like "and" or "or" and phrases that indicate cross-referencing needs.

The **Retriever Node** performs hybrid retrieval by querying both BM25 and FAISS indexes. It retrieves top-k chunks from each, normalizes scores to a $[0, 1]$ range. Retrieved chunks go into the workflow state.

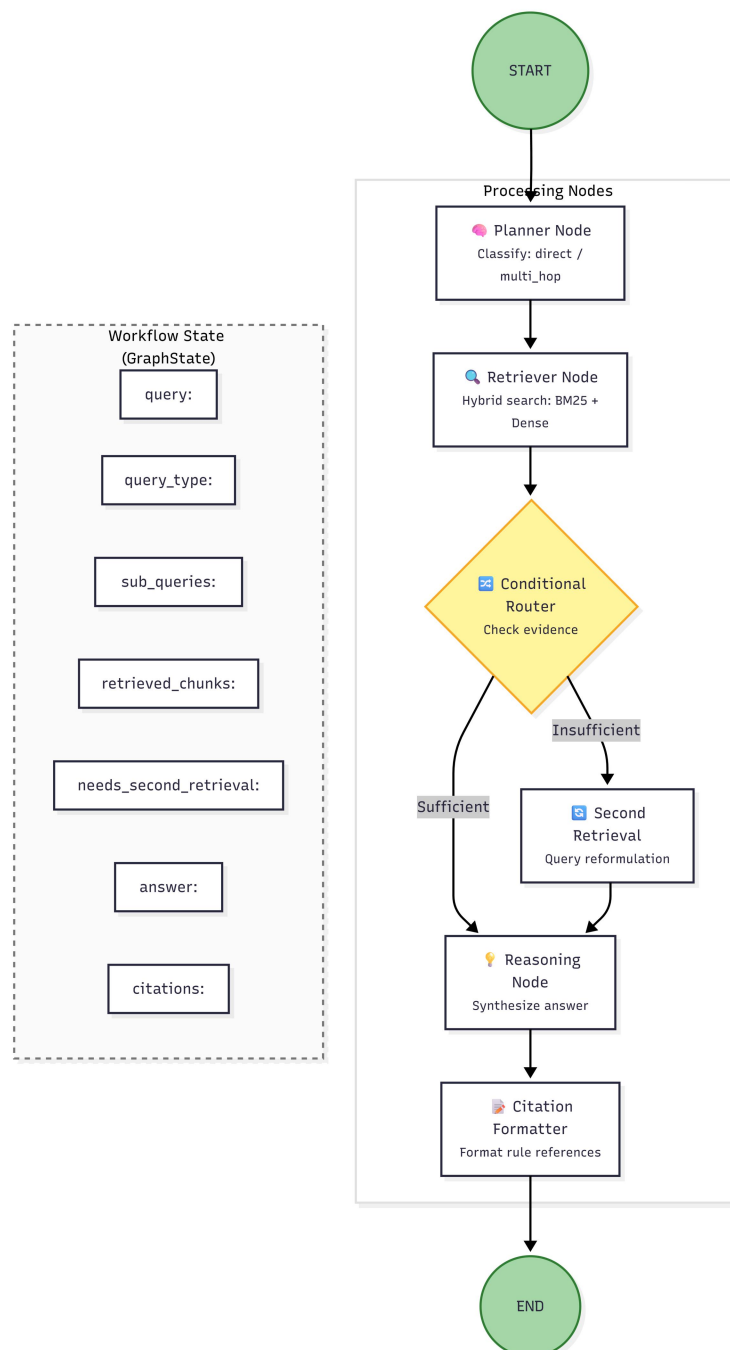
The **Conditional Router** checks whether the retrieved evidence covers all sub-queries. Specifically, for each sub-query generated by the Planner, the router checks whether at least one retrieved chunk has a relevance score above a minimum threshold (0.3 in Phase 1). If any sub-query lacks supporting evidence, the router sets `needs_second_retrieval = True` and sends the workflow to the **Second Retrieval Node**. For `direct` queries, the router always proceeds to reasoning.

The **Second Retrieval Node** performs another retrieval pass using the uncovered sub-queries as new search terms. Newly retrieved chunks are appended to the state, deduplicated by `chunk_id`.

The **Reasoning Node** uses DeepSeek Reasoner to synthesize an answer based on all retrieved chunks. The prompt instructs the LLM to ground every claim in the retrieved evidence and include an uncertainty note if evidence is incomplete.

The **Citation Formatter Node** post-processes the answer to ensure citations are properly formatted with rule numbers, chapter references, and source document identifiers. The final structured response is returned to the user.

Figure 2: LangGraph Agent Workflow



3.4 Design Justifications

Several key design choices shaped the architecture.

Structure-Aware Chunking: We use structure-aware chunking instead of naive token-based splitting because regulatory documents have inherent hierarchical structure. A rule provision often spans multiple paragraphs. Splitting it arbitrarily would break the logical flow. By preserving rule numbers and section titles in chunk metadata, we enable more precise retrieval and citation generation. Users can also trace answers back to the original document structure more easily [12].

Hybrid Retrieval: We combine BM25 and dense embeddings instead of using only dense retrieval. As discussed in Section 2.2, dense retrieval excels at semantic matching but can retrieve chunks that are semantically similar yet legally imprecise. BM25 ensures that chunks with exact keyword matches are not overlooked. The hybrid approach balances recall with precision, which is particularly important for regulatory text where specific rule numbers and legal terms carry precise meaning.

LangGraph for Workflow Orchestration: We chose LangGraph over a hardcoded pipeline because it supports conditional branching and stateful execution [18]. This matters for the conditional second retrieval step, which only triggers when initial retrieval is insufficient. LangGraph also makes the workflow easier to extend. Adding new nodes (like a tool-calling node in Phase 2) requires minimal changes to the existing graph structure.

Lightweight Agentic Approach: For Phase 1, we kept the agentic design simple. The planner uses heuristic classification rather than full LLM-driven reasoning. The system does not yet support tool use or session memory. This reduces complexity and allows us to focus on validating the core workflow. A more sophisticated multi-agent system with specialized sub-agents for different rule categories would be more powerful, but would also introduce additional failure modes and debugging challenges that are not justified at the prototype stage.

4. Methodology and Algorithms

Following the system architecture presented in Section 3, this section details the specific algorithms and methodologies used to implement each component.

4.1 Knowledge Base Construction Pipeline

The ingestion pipeline focuses on maintaining structural fidelity when converting regulatory documents. The process follows a deterministic sequence:

- 1. Format Handling:** The `DocumentLoader` employs regex-based pattern matching to differentiate between rule headers, section titles, and body paragraphs. This allows the system to support `.txt`, `.md`, and basic `.pdf` (via text-layer extraction) formats.
- 2. Text Normalization:** The `DocumentCleaner` applies standard preprocessing: encoding conversion to UTF-8, removal of non-breaking spaces, and normalization of line endings (`\n` vs `\r\n`).
- 3. Structure-Aware Chunking:** Unlike standard splitters, our `Chunker` implements a hierarchical parsing strategy that respects the HKEX rulebook's structure [12].
- 4. Metadata Mapping:** Metadata is extracted during the parsing phase and mapped to a JSON schema before storage. Chunks are persisted in `data/chunks/` as JSON files, enabling efficient downstream loading for indexing.

4.2 Structure-Aware Chunking Algorithm

The chunking algorithm avoids naive token limits by prioritizing logical document boundaries.

```
def structure_aware_chunk(document):  
  
    hierarchy = parse_hierarchy(document) # Uses regex for Chapter/Rule patterns  
  
    chunks = []  
  
    for section in hierarchy:  
  
        # Avoid splitting if section is within size constraints  
  
        if len(section.text) <= MAX_CHUNK_LENGTH:
```

```
        chunks.append(create_chunk(section))

    else:

        # Recursive split at natural paragraph breaks

        chunks.extend(recursive_split(section, MAX_CHUNK_LENGTH))

    return enrich_metadata(chunks)
```

Splitting text by token count often fails in legal documents. It might cut a rule in half or separate it from its context. We use a structure-aware approach instead:

1. Parse Hierarchy: The system looks for patterns like "Chapter 14A", section headings, and rule numbers (e.g., "14A.35").
2. Context Preservation: Each chunk is kept as a self-contained unit with its rule number and section context.
3. Split Logic: If a rule is too long (over 512 tokens), the system splits it at natural breaks like paragraphs or lists, rather than at fixed character counts.
4. Metadata: Metadata is extracted during the parsing phase and mapped to a JSON schema before storage. Chunks are persisted in `data/chunks/` as JSON files, enabling efficient downstream loading for indexing.

4.3 Hybrid Retrieval Strategy

We combine lexical and semantic search to maximize retrieval robustness [13, 14].

1. BM25: Used for exact keyword matching, prioritizing rule-specific terminology.
2. Dense Retrieval: We use BGE-M3 embeddings via Ollama [16] to capture semantic intent.
3. Score Fusion: Scores are normalized to the [0, 1] range using Min-Max scaling before weighted fusion:

$$Score_{final} = w_1 \cdot Score_{bm25} + w_2 \cdot Score_{dense}$$

4. Deduplication: Finally, we filter by `chunk_id` to remove duplicate results returned by both indices.

4.4 Agentic Workflow: Planning, Retrieval and Reasoning

The system uses LangGraph to manage the agent workflow. The process runs in a loop:

1. Planner: The `PlannerNode` decides if the query is `direct` or `multi_hop`. If `multi_hop`, it breaks the question into sub-queries.
2. Retriever: The `RetrieverNode` searches both BM25 and FAISS indexes and keeps the results in the state.
3. Router: The `RouterNode` decides if a second search is needed. It checks if the current chunks answer all sub-queries (with a relevance score over 0.3).
4. Second Retrieval: If needed, the `SecondRetrievalNode` reformulates the query to fill in the missing information.
5. Reasoning: The `ReasoningNode` uses DeepSeek Reasoner to synthesize an answer from all retrieved evidence.
6. Citation: The `CitationFormatterNode` adds rule numbers and source references to the answer.

4.5 Current Limitations and Planned Enhancements

As an MVP, the system has limitations that will be addressed in Phase 2:

1. Heuristic Planner: Currently rule-based; we intend to shift to LLM-driven planning for better query understanding.
2. Evidence Coverage: Second retrieval is triggered by simple relevance thresholds; this will be upgraded to an evidence-coverage checking model [9].
3. Tool Integration: While the interface is implemented, we have not yet integrated external tools such as financial calculators.

4. Memory and Context: The system is stateless. Adding conversational memory will be a priority.
5. Formal Evaluation: Rigorous evaluation using RAGAS is scheduled for the next phase.

5. Preliminary Performance Analysis or Experiments

5.1 Experimental Setup

This section reports early-stage feasibility testing, not a comprehensive benchmark evaluation. The goal is to verify that the core pipeline works end-to-end and produces reasonable outputs.

Table 2: Experimental Environment

Component	Specification
OS	Windows 11
CPU	AMD Ryzen 7
RAM	16 GB
GPU	Not required
Python	3.13.5
Backend	FastAPI 0.135.1, Uvicorn 0.41.0
Workflow	LangGraph 1.0.10, LangChain 1.2.10
Lexical Index	rank-bm25 0.2.2
Vector Index	FAISS 1.13.2
Embeddings	BGE-M3 via Ollama
LLM Client	DeepSeek Reasoner API
Validation	Pydantic 2.11.7
Testing	pytest 8.3.4
Test Documents	HKEX Main Board Listing Rules excerpts

The test document set consists of excerpts rather than the full consolidated rulebook. This is sufficient for validating the pipeline but does not represent the scale of a production deployment.

5.2 Functional Verification

We maintain 68 unit and integration tests across 8 test files. These cover the following modules:

Table 3: Test Coverage Summary

Test File	Module	Tests	Status
<code>test_chunker.py</code>	Structure-aware chunking	12	Pass

<code>test_cleaner.py</code>	Text cleaning	6	Pass
<code>test_hybrid_retrieval.py</code>	BM25 + Dense + Fusion	14	Pass
<code>test_planner.py</code>	Query classification	8	Pass
<code>test_planner_refactor.py</code>	LLM route planner + validators	14	Pass
<code>test_chat_api.py</code>	API endpoint	4	Pass
<code>test_stage1_agentic_components.py</code>	Agentic components	6	Pass
<code>test_integration_v2.py</code>	End-to-end workflow	4	Pass

Key verifications include: documents are ingested and chunks are generated with correct metadata fields, BM25 and FAISS indexes build without errors, the hybrid retriever returns ranked results with fused scores, the API endpoint accepts queries and returns structured JSON responses, and citations in the output are traceable to specific chunk IDs and rule numbers.

6. Milestones and Overall Schedule

6.1 Work Completed So Far

Phase 1 focused on building a functional backend prototype. Our work can be summarized in three areas:

- 1. Knowledge Base Construction:** We built a document ingestion pipeline that loads, cleans, and chunks HKEX regulatory documents while preserving their hierarchical structure. The pipeline produces dual indexes: BM25 for lexical matching and FAISS with BGE-M3 embeddings for semantic retrieval.
- 2. Agentic Retrieval & Reasoning:** We implemented a LangGraph-based workflow with a heuristic planner, hybrid retriever, conditional router, and reasoning agent powered by DeepSeek Reasoner. The system classifies queries as `direct` or `multi_hop`, triggers conditional second retrieval, and generates citation-grounded answers. We also developed Stage 1 enhancements including an LLM-driven route planner, task decomposer, and validation components.
- 3. System Integration & Testing:** We delivered a FastAPI backend with `/health` and `/chat` endpoints, supported by 68 unit and integration tests covering ingestion, retrieval, agentic nodes, and API functionality. Project documentation includes design specifications, usage guides, and this interim report.

6.2 Project Schedule and Milestones

We are moving from the Phase 1 prototype to Phase 2, where we focus on reasoning quality, specialized tools, and formal evaluation.

Table 4: Project schedule

Phase	Milestone	Expected Completion	Status
Phase 1	Background Research & Technology Selection	February 2025	Completed
Phase 1	Backend Prototype & Core Workflow	March 2025	Completed
Phase 1	LLM-driven Planner & Coverage Check	April 2025	Pending
Phase 2	Tool Integration (Calculators)	April 2025	Pending
Phase 2	Benchmarking & RAGAS Evaluation	May 2025	Pending

Phase 3 Frontend & Production Prep

June 2025

Pending

7. Work to be Completed for the Next Report

7.1 Technical Improvements

Following the schedule outlined in Table 4, the next phase addresses the limitations identified in Section 4.5 and Section 5.2. Here we outline the concrete implementation plan.

Planner Enhancement: The current heuristic planner will be replaced with an LLM-driven intent classifier that can better distinguish between direct lookups, multi-hop reasoning, and tool-dependent queries. This includes integrating the task decomposer and validation components developed in Stage 1 into the main workflow.

Evidence Coverage Verification: A coverage checker is planned that verifies whether retrieved chunks actually address all aspects of a multi-hop query before generating an answer. This will reduce cases where the system produces incomplete or surface-level responses.

Answer Verification: A post-generation verification step will check whether the generated answer is grounded in the retrieved evidence and flag potential hallucinations or unsupported claims.

Tool Integration: The tool interface will be implemented for specialized compliance tasks, starting with a Size Test Calculator that can compute percentage ratios from structured financial inputs, and a Rule Lookup Tool that can fetch specific provisions on demand.

Retrieval Improvements: The score fusion strategy will be upgraded from weighted linear combination to Reciprocal Rank Fusion (RRF), which is more robust to score distribution differences. Query rewriting and reranking will also be explored to improve retrieval precision for ambiguous queries.

7.2 Evaluation and Reporting Work

Formal evaluation is essential for validating the system's performance in a regulatory compliance context.

Benchmark Dataset: A benchmark dataset with 20-30 annotated compliance questions will be constructed, covering direct lookups, multi-hop reasoning, and tool-dependent scenarios. Each question will include ground-truth answers and expected citations.

Evaluation Metrics: Metrics will be implemented for retrieval recall (whether the correct chunks are retrieved), citation quality (whether citations are accurate and traceable), and answer correctness (whether the generated answer matches the ground truth). Response time and system reliability will also be measured.

Baseline Comparison: The agentic RAG system will be compared against baseline approaches, including standard single-pass RAG and keyword-based search, to demonstrate the value of the agentic workflow.

System Demonstration: A recorded demonstration will be prepared showing the system handling representative compliance queries, including cases where it correctly triggers second retrieval, uses tools, and flags uncertainty.

Final Report: The final report will include comprehensive results, error analysis using RAGAS [9], and a discussion of the system's capabilities and limitations as a research prototype.

8. References

- [1] Hong Kong Exchanges and Clearing Limited, "Market highlights," 2025. [Online]. Available: <https://www.hkex.com.hk/Market-Data/Statistics>
- [2] Hong Kong Exchanges and Clearing Limited, "Main Board Listing Rules," consolidated version. [Online]. Available: <https://en-rules.hkex.com.hk/rulebook/main-board-listing-rules>
- [3] Hong Kong Exchanges and Clearing Limited, "Rules & resources — Guidance." [Online]. Available: <https://www.hkex.com.hk/Listing/Rules-and-Resources/Guidance>
- [4] H. Zhong et al., "Legal retrieval and question answering: A survey," arXiv preprint arXiv:2210.13314, 2022.
- [5] L. Wang et al., "A survey on large language model based autonomous agents," arXiv preprint arXiv:2308.11432, 2023.
- [6] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [7] Y. Shao et al., "Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy," in *Findings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [8] D. M. Katz, "Legal tech, smart contracts and blockchain," in *Perspectives on Law and Innovation*. Cheltenham, UK: Edward Elgar Publishing, 2018.
- [9] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," arXiv preprint arXiv:2309.15217, 2023.
- [10] H. Zhong et al., "JEC-QA: A legal-domain question answering dataset," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 5, pp. 9701–9708, 2020.
- [11] R. Susskind, *Tomorrow's Lawyers: An Introduction to Your Future*. Oxford, UK: Oxford Univ. Press, 2017.
- [12] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge Univ. Press, 2008.

- [13] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. 2020 Conf. Empirical Methods Nat. Lang. Process.*, 2020, pp. 6769–6781.
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.
- [15] C. Xiao et al., "C-Pack: Packaged resources to advance general Chinese embedding," arXiv preprint arXiv:2309.07597, 2023.
- [16] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 9459–9474.
- [17] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2023.
- [18] LangChain, "LangGraph: A framework for building stateful agent workflows," 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph/>